AI+X 중급 프로젝트
중간점검발표 (2)

202310843 송예린
202310847 심영주
202310856 유세리

논문

문제점 발견

Process-aware Human Activity Recognition

Jiawei Zheng, Petros Papapanagiotou, Jacques D. Fleuriot, Jane Hillston

School of Informatics, University of Edinburgh
jw.zheng@ed.ac.uk, petrospapapan@gmail.com, jdf@ed.ac.uk, jane.hillston@ed.ac.uk

Abstract

ıs naturally follow distinct patterns when conducting
aily activities, which are driven by established prac-
nd processes, such as production workflows, social
and daily routines. Human activity recognition (HAR)
ıms usually use neural networks or machine learning
ques to analyse inherent relationships within the data.

To address this limitation, the idea of incorporati
main knowledge or contextual information into ML a
methods has been investigated to improve the accura
interpretability (Guo et al. 2023). Humans inherently
distinct patterns in their daily activities, influenced by
lished best practices and workflows. These include p
tion processes, social norms and habitual routines, s

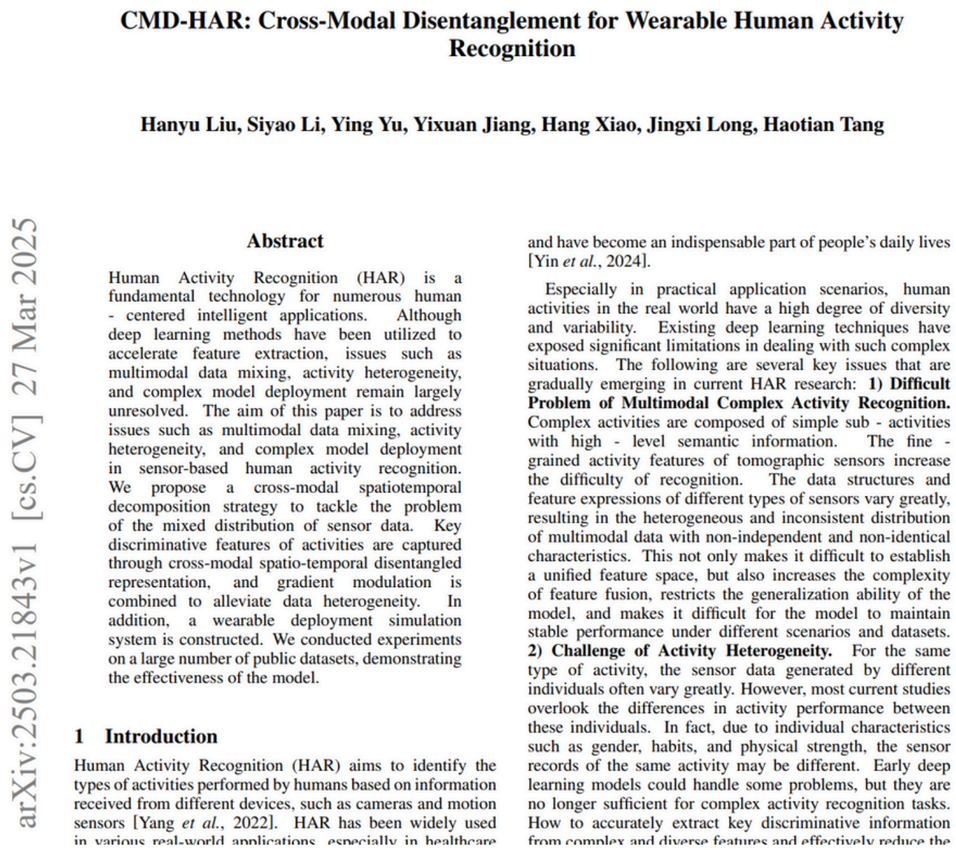
기존 논문

해당 논문에서 사용한 데이터셋

- EatSense Dataset
- 수집방법: RGB-D 카메라로 사람들의 식사 모습 촬용
- 센서: Skeleton keypoints(코, 어깨 등)
- 활동 범주: 식사 행동 중심

=> UCI HAR을 사용하는
더 적합한 논문을 찾게 됨

CMD-HAR: Cross-Modal Disentanglement for Wearable Human Activity Recognition



CMD-HAR: Cross-Modal Disentanglement for Wearable Human Activity Recognition

Hanyu Liu, Siyao Li, Ying Yu, Yixuan Jiang, Hang Xiao, Jingxi Long, Haotian Tang

arXiv:2503.21843v1 [cs.CV] 27 Mar 2025

Abstract

Human Activity Recognition (HAR) is a fundamental technology for numerous human - centered intelligent applications. Although deep learning methods have been utilized to accelerate feature extraction, issues such as multimodal data mixing, activity heterogeneity, and complex model deployment remain largely unresolved. The aim of this paper is to address issues such as multimodal data mixing, activity heterogeneity, and complex model deployment in sensor-based human activity recognition. We propose a cross-modal spatiotemporal decomposition strategy to tackle the problem of the mixed distribution of sensor data. Key discriminative features of activities are captured through cross-modal spatio-temporal disentangled representation, and gradient modulation is combined to alleviate data heterogeneity. In addition, a wearable deployment simulation system is constructed. We conducted experiments on a large number of public datasets, demonstrating the effectiveness of the model.

1 Introduction

Human Activity Recognition (HAR) aims to identify the types of activities performed by humans based on information received from different devices, such as cameras and motion sensors [Yang *et al.*, 2022]. HAR has been widely used in various real-world applications, especially in healthcare and have become an indispensable part of people's daily lives [Yin *et al.*, 2024].

Especially in practical application scenarios, human activities in the real world have a high degree of diversity and variability. Existing deep learning techniques have exposed significant limitations in dealing with such complex situations. The following are several key issues that are gradually emerging in current HAR research: **1) Difficult Problem of Multimodal Complex Activity Recognition.** Complex activities are composed of simple sub - activities with high - level semantic information. The fine - grained activity features of tomographic sensors increase the difficulty of recognition. The data structures and feature expressions of different types of sensors vary greatly, resulting in the heterogeneous and inconsistent distribution of multimodal data with non-independent and non-identical characteristics. This not only makes it difficult to establish a unified feature space, but also increases the complexity of feature fusion, restricts the generalization ability of the model, and makes it difficult for the model to maintain stable performance under different scenarios and datasets. **2) Challenge of Activity Heterogeneity.** For the same type of activity, the sensor data generated by different individuals often vary greatly. However, most current studies overlook the differences in activity performance between these individuals. In fact, due to individual characteristics such as gender, habits, and physical strength, the sensor records of the same activity may be different. Early deep learning models could handle some problems, but they are no longer sufficient for complex activity recognition tasks. How to accurately extract key discriminative information from complex and diverse features and effectively reduce the

센서 기반 인간 행동 인식의 성능을 저해하는 주요 문제들을 해결하기 위해 새로운 프레임워크 CMD-HAR을 제안

센서 데이터에서 시공간 특성을 분리하고
Gradient를 조절하여 여러 센서의 학습 기여도 불균형을 해결함

총 6개의 데이터셋 사용

- WISDM
- UCI-HAR
- PAMAP2
- USC-HAD
- OPPORTUNITY
- UniMiB-SHAR

데이터셋	수집방법	센서	주요활동
UCI-HAR	스마트폰을 허리에 장착	Accelerometer, Gyroscope	걷기, 앉기, 누워있기 등
USC-HAD	허리 착용 센서로 일상 행동 측정	Accelerometer	걷기, 점프, 앉기 등
WISDM	스마트폰을 허리에 착용	Accelerometer, Gyroscope	걷기, 뛰기, 앉기, 계단 오르내리기 등

UCI-HAR 가공 방법

1. 슬라이딩 윈도우 적용

- 샘플당 약 2.56초 분량

2. 데이터 분할 (Train / Validation / Test)

- 8:1:1 비율

3. 모달리티 분리 및 채널 정리

- 가속도와 자이로스코프 데이터를 각각 별도 그룹으로 구분
- 각 센서 축을 개별 채널로 처리해 총 6채널

4. 정규화 (Normalization)

- 각 채널에 Z-score 정규화를 적용

5. 평가지표

- Accuracy, F1-score, G-mean 세 가지 사용

스마트폰 센서 데이터 단계별 모델링
3번 파일

2.데이터 전처리

- 세부 요구사항
  - Label 추가 : data 에 Activity_dynamic 를 추가합니다. Activity_dynamic은 과제1에서 is_dynamic과 동일한 값입니다.
  - x와 y1, y2로 분할하시오.
    - y1 : Activity
    - y2 : Activity_dynamic
  - train : val = 8 : 2 혹은 7 : 3
  - random_state 옵션을 사용하여 다른 모델과 비교를 위해 성능이 재현되도록 합니다.

```python
data['Activity_dynamic'] = data['Activity'].map({'WALKING':1,'WALKING_UPSTAIRS':1,'WALKING_DOWNSTAIRS':1,'LAYING':0,'STANDING':0,'SITTING':0})
data.head()
```

Python

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	tBodyAcc-max()-X	...	angle(tBodyAccMean,gravity)	angle(tBodyAccJerkMean),g
0	0.288508	-0.009196	-0.103362	-0.988986	-0.962797	-0.967422	-0.989000	-0.962596	-0.965650	-0.929747	...	-0.042494	
1	0.265757	-0.016576	-0.098163	-0.989551	-0.994636	-0.987435	-0.990189	-0.993870	-0.987558	-0.937337	...	-0.062899	
2	0.278709	-0.014511	-0.108717	-0.997720	-0.981088	-0.994008	-0.997934	-0.982187	-0.995017	-0.942584	...	0.000265	
3	0.289795	-0.035536	-0.150354	-0.231727	-0.006412	-0.338117	-0.273557	0.014245	-0.347916	0.008288	...	-0.255125	
4	0.394807	0.034098	0.091229	0.088489	-0.106636	-0.388502	-0.010469	-0.109680	-0.346372	0.584131	...	-0.044344	

5 rows × 564 columns

```python
X = data.drop(['Activity', 'Activity_dynamic'], axis = 1)
y1 = data['Activity']
y2 = data['Activity_dynamic']
```

Python

```python
X_train, X_val, y_train, y_val = train_test_split(X, y2, test_size = 0.2, random_state = 42, shuffle=False)
X_train.info()
```

Python

```
<class 'pandas.core.frame.DataFrame'>
Index: 4704 entries, 0 to 4703
Columns: 562 entries, tBodyAcc-mean()-X to subject
dtypes: float64(561), int64(1)
memory usage: 20.2 MB
```

(1) 단계1 : 정적/동적 행동 분류 모델

- 세부 요구사항
  - 정적 행동(Laying, Sitting, Standing)과 동적 행동(동적 : Walking, Walking-Up, Walking-Down)을 구분하는 모델 생성.
  - 몇가지 모델을 만들고 가장 성능이 좋은 모델을 선정하시오.

1) 알고리즘1 : XGBClassifier

```python
from xgboost import XGBClassifier

xgb_model = XGBClassifier(random_state=42)
xgb_model.fit(X_train, y_train)

pred = xgb_model.predict(X_val)

#평가
print('accuracy :',accuracy_score(y_val, pred))
print('='*60)
print(confusion_matrix(y_val, pred))
print('='*60)
print(classification_report(y_val, pred))

# F1 스코어 확인 (micro, macro, weighted 등 선택 가능)
f1_xgb = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_xgb)
```

```
accuracy : 1.0
============================================================
[[658   0]
 [  0 519]]
============================================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       658
           1       1.00      1.00      1.00       519

    accuracy                           1.00      1177
   macro avg       1.00      1.00      1.00      1177
weighted avg       1.00      1.00      1.00      1177

F1 Score (weighted): 1.0
```

2) 알고리즘2 : RandomForestClassifier

```python
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

pred = rf_model.predict(X_val)
print('accuracy :', accuracy_score(y_val, pred))
print('='*60)
print('Confusion Matrix:')
print(confusion_matrix(y_val, pred))
print('='*60)
print('Classification Report:')

# F1 스코어 확인 (weighted 평균)
f1_rf = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_rf)
```

```
accuracy : 0.9991503823279524
============================================================
Confusion Matrix:
[[657   1]
 [  0 519]]
============================================================
Classification Report:
F1 Score (weighted): 0.9991504681425801
```

(2) 단계2-1 : 정적 동작 세부 분류

- 세부 요구사항
  - 정적 행동(Laying, Sitting, Standing)인 데이터 추출
  - Laying, Sitting, Standing 를 분류하는 모델을 생성
  - 몇가지 모델을 만들고 가장 성능이 좋은 모델을 선정하시오.

```python
data_static = data[data['Activity_dynamic'] == 0]
data_static.info()
```

Python

```
<class 'pandas.core.frame.DataFrame'>
Index: 3234 entries, 0 to 5880
Columns: 564 entries, tBodyAcc-mean()-X to Activity_dynamic
dtypes: float64(561), int64(2), object(1)
memory usage: 13.9+ MB
```

```python
X_static = data_static.drop(['Activity', 'Activity_dynamic'], axis = 1)
y1_static = data_static['Activity'].map({'LAYING':0, 'STANDING':1, 'SITTING':2})
```

Python

```python
X_train, X_val, y_train, y_val = train_test_split(X_static, y1_static, test_size = 0.2, random_state = 42, shuffle=False)
```

Python

XGBoost

```python
from xgboost import XGBClassifier

xgb_model = XGBClassifier(random_state=42)
xgb_model.fit(X_train, y_train)

pred = xgb_model.predict(X_val)

#평가
print('accuracy :',accuracy_score(y_val, pred))
print('='*60)
print(confusion_matrix(y_val, pred))
print('='*60)
print(classification_report(y_val, pred))

# F1 스코어 확인 (micro, macro, weighted 등 선택 가능)
f1_xgb = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_xgb)
```

```
accuracy : 0.990726429675425
============================================================
[[229   0   0]
 [  0 214   3]
 [  0   3 198]]
============================================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       229
           1       0.99      0.99      0.99       217
           2       0.99      0.99      0.99       201

    accuracy                           0.99       647
   macro avg       0.99      0.99      0.99       647
weighted avg       0.99      0.99      0.99       647

F1 Score (weighted): 0.990726429675425
```

RandomForest

```python
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

pred = rf_model.predict(X_val)
print('accuracy :', accuracy_score(y_val, pred))
print('='*60)
print('Confusion Matrix:')
print(confusion_matrix(y_val, pred))
print('='*60)
print('Classification Report:')

# F1 스코어 확인 (weighted 평균)
f1_rf = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_rf)
```

```
accuracy : 0.9706336939721792
============================================================
Confusion Matrix:
[[229   0   0]
 [  0 211   6]
 [  0  13 188]]
============================================================
Classification Report:
F1 Score (weighted): 0.9706065521181049
```

SVM

```python
from sklearn.svm import SVC

svm_model = SVC(random_state=42)
svm_model.fit(X_train, y_train)

pred = svm_model.predict(X_val)

# 평가
print('accuracy :', accuracy_score(y_val, pred))
print('='*60)
print(confusion_matrix(y_val, pred))
print('='*60)
print(classification_report(y_val, pred))

# F1 스코어 확인 (weighted 평균)
f1_svm = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_svm)
```

```
accuracy : 0.8979907264296755
============================================================
[[229   0   0]
 [  0 193  24]
 [  2  40 159]]
============================================================
              precision    recall  f1-score   support

           0       0.99      1.00      1.00       229
           1       0.83      0.89      0.86       217
           2       0.87      0.79      0.83       201

    accuracy                           0.90       647
   macro avg       0.90      0.89      0.89       647
weighted avg       0.90      0.90      0.90       647

F1 Score (weighted): 0.8973651477648605
```

KNN

```python
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)

pred = knn_model.predict(X_val)

# 평가
print('accuracy :', accuracy_score(y_val, pred))
print('='*60)
print(confusion_matrix(y_val, pred))
print('='*60)
print(classification_report(y_val, pred))

# F1 스코어 확인 (weighted 평균)
f1_knn = f1_score(y_val, pred, average='weighted')
print('F1 Score (weighted):', f1_knn)
```

```
accuracy : 0.9335394126738794
============================================================
[[229   0   0]
 [  0 203  14]
 [  1  28 172]]
============================================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       229
           1       0.88      0.94      0.91       217
           2       0.92      0.86      0.89       201

    accuracy                           0.93       647
   macro avg       0.93      0.93      0.93       647
weighted avg       0.93      0.93      0.93       647

F1 Score (weighted): 0.9332673972044596
```

모델 비교

```python
print('\n[ F1 Score Summary (weighted)]:')
print(f'XGBoost : {f1_xgb:.4f}')
print(f'Random Forest : {f1_rf:.4f}')
print(f'SVM : {f1_svm:.4f}')
print(f'KNN : {f1_knn:.4f}')

best_model = max(
    [('XGBoost', f1_xgb),
     ('Random Forest', f1_rf),
     ('SVM', f1_svm),
     ('KNN', f1_knn)],
    key=lambda x: x[1]
)

print(f'\nBest model: {best_model[0]} with F1 Score: {best_model[1]:.4f}')
```

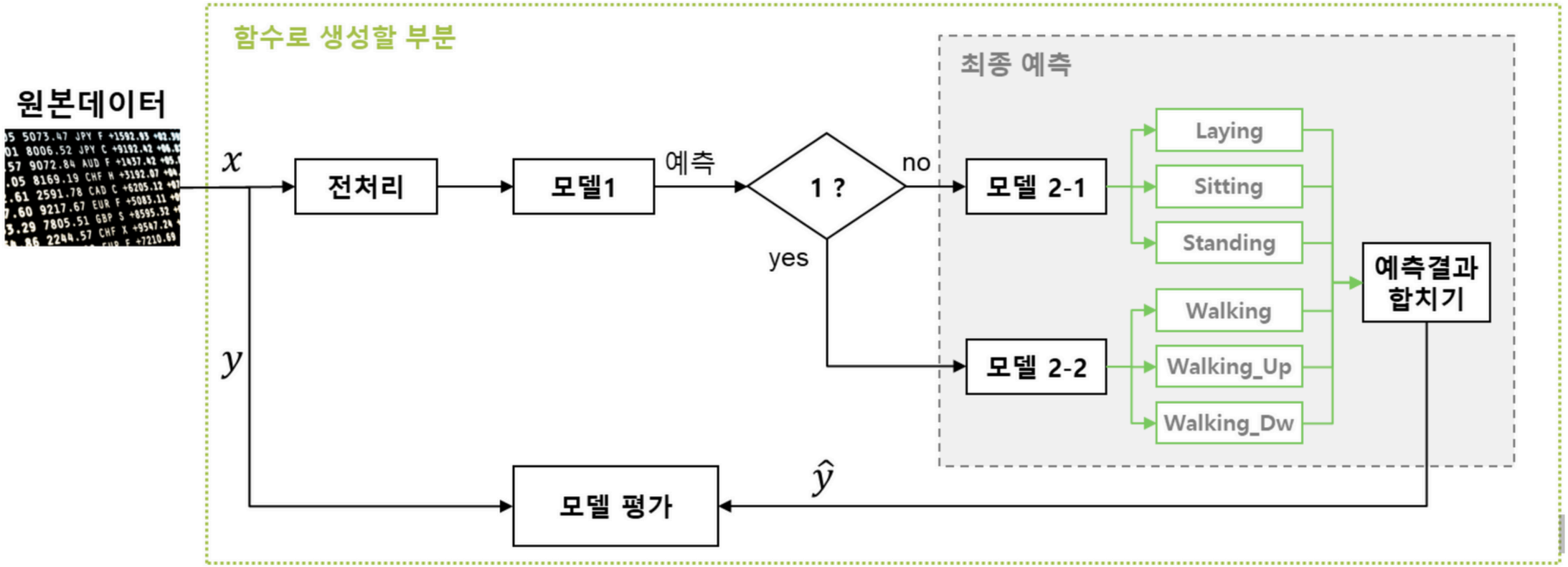
```
[ F1 Score Summary (weighted)]:
XGBoost : 0.9907
Random Forest : 0.9706
SVM : 0.8974
KNN : 0.9333

Best model: XGBoost with F1 Score: 0.9907
```

(3) 단계2-2 : 동적 동작 세부 분류

- 세부 요구사항
  - 동동적 행동(Walking, Walking Upstairs, Walking Downstairs)인 데이터 추출
  - Walking, Walking Upstairs, Walking Downstairs 를 분류하는 모델을 생성
  - 몇가지 모델을 만들고 가장 성능이 좋은 모델을 선정하시오.

(4) 분류 모델 합치기

- 세부 요구사항
  - 두 단계 모델을 통합하고, 새로운 데이터(test)에 대해서 최종 예측결과와 성능평가가 나오도록 함수로 만들기
  - 데이터 파이프라인 구축 : test데이터가 로딩되어 전처리 과정을 거치고, 예측 및 성능 평가 수행



+Hyperparameter tunning